PRODUCTION SYSTEM ARCHITECTURE & ENGINEERING MANUAL

NexusResearch — Complete Technical Architecture, Development & Reconstruction Guide

How to Understand, Run, Rebuild, Maintain and Deploy the Complete Real-Time AI Research Platform from Scratch Without AI

Author & System Architect:	Tejeshwar Divekar
Project Repository:	https://github.com/TejeshwarDivekar/NexusAI.git
Live Production Frontend:	https://frontend-production-2df5.up.railway.app
Live Production API:	https://backend-production-873b.up.railway.app/api/v1/health
Architecture Version:	1.0.0-Production (Next.js 16.3.1 + FastAPI 1.0.0 + Gemini 2.5)
Release Date:	August 21, 2026

Table of Contents

1. Project Overview & Executive Summary

NexusResearch is an enterprise-grade, autonomous real-time research assistant and academic intelligence workspace. Unlike conventional LLM wrappers that merely generate plausible-sounding text, NexusResearch implements an evidence-grounded, multi-stage verification pipeline. It retrieves authentic peer-reviewed literature, live web disclosures, and uploaded user documents, subjects all candidate sources to a strict hard relevance gate, maps exact quote-level evidence into a claim matrix, and synthesizes structured, publication-grade intelligence with interactive citations and professional IEEE Word (.docx) document generation.

Core Mandate: Source Validity (registry authentication) is strictly separated from Query Relevance (topical alignment). Candidate papers are evaluated against a multi-factor relevance scorer with hard negative disqualification before synthesis begins.

2. What NexusResearch Does: Core Capabilities

• Academic & Scientific Deep Dives: Executes multi-query expansion across OpenAlex, arXiv, PubMed, Europe PMC, and Crossref. Synthesizes formal state-of-the-art reports with mathematical formulations, methodologies, and benchmarking data.
• Real-Time & Live Market Intelligence: Fetches real-time web data via DuckDuckGo and Wikipedia with live timestamp verification for breaking news, stock metrics, and technology releases.
• Step-by-Step Roadmaps: Generates structured milestone tracks (Foundational -> Intermediate -> Advanced) with concrete prerequisites, toolsets, and project milestones.
• Comparative Analyses: Constructs side-by-side dimensional matrices evaluating trade-offs, performance benchmarks, and cost profiles.
• Numerical Calculations: Executes deterministic Python calculations (mean, standard deviation, growth rates) alongside scientific analysis.

3. Core Features & Functional Architecture

1. Answer-First Research Workspace: Desktop 3-panel workspace (Sources | Main Answer | Evidence Matrix) and Purpose-Built Mobile Experience with segmented navigation and 44px+ touch targets.
2. Multi-Signal Source Relevance Scorer: Multi-factor heuristic scoring engine with negative domain disqualification and hard gating (threshold >= 0.48).
3. Evidence Matrix & Claim Grounding: Direct mapping from assertions to exact source excerpts, why-relevant rationales, and qualitative confidence scores.
4. Contradiction & Consensus Engine: Identifies conflicting claims and methodological divergences across literature.
5. IEEE Two-Column DOCX Document Builder: Generates fully compliant IEEE Word documents ready for publication.
6. NextAuth v5 & User Isolation: Secure Google/GitHub OAuth authentication with PostgreSQL user-level project isolation.
7. Dynamic Next.js API Proxying: Edge route proxy at /api/v1/[...path] forwarding requests with 120s timeout and binary streaming support.

4. Complete Technology Stack & Version Inventory

• Next.js: v16.3.1 (App Router, Server Actions, Route Proxying)
• React & React-DOM: v19.2.8 (Component tree, hooks, hydration)
• Tailwind CSS: v4.0.0 (CSS-first token architecture, dark mode)
• NextAuth: v5.0.0-beta.32 (Google/GitHub OAuth, JWT sessions)
• FastAPI: >=0.110.0 (High-performance async REST API)
• Uvicorn: >=0.28.0 (Production async ASGI server)
• SQLAlchemy: >=2.0.28 (Database ORM & automated migration)
• Psycopg2-Binary: >=2.9.9 (PostgreSQL high-performance driver)
• Pydantic: >=2.6.0 (Request/Response data validation schemas)
• HTTPX: >=0.27.0 (Async HTTP search requests)
• Python-Docx: >=1.2.0 (Programmatic IEEE DOCX compilation)
• Google Gemini 2.5: REST / SDK (Query classification, synthesis & grounding)

5. Why Each Technology Is Used

• Next.js 16 + React 19: Provides high-speed App Router server rendering, dynamic streaming capabilities, and built-in API proxy routing without UI flicker.
• FastAPI + Python 3.10+: Native asynchronous I/O (asyncio) allows concurrent multi-source querying across 5+ academic search providers in parallel.
• PostgreSQL + SQLAlchemy 2.0: Relational integrity ensures rigorous mapping between Users, Projects, Research Tasks, Claims, Evidence, and Sources.
• Google Gemini 2.5: 1M+ token context window and superior JSON structured output compliance for deterministic claim-evidence mapping.
• Python-Docx: Allows complete programmatic control over binary Word XML packaging to build IEEE standard manuscripts directly on the server.

6. Complete System Architecture & Communication Flow

Client Browser -> Next.js 16 Edge / App Router (Frontend) -> Reverse Proxy (/api/v1/[...path]) -> FastAPI Backend -> ResearchEngine Orchestrator -> Search Providers (OpenAlex, arXiv, PubMed, Crossref, DDG) -> SourceRelevanceScorer (Hard Gate >= 0.48) -> EvidenceService -> ContradictionService -> Gemini 2.5 LLM -> IEEEDocumentGenerator (.docx) -> PostgreSQL Persistence -> Client Streaming Response.

7. Full Project Directory & File Manifest

backend/app/main.py (FastAPI Bootstrap), backend/app/routers/ (auth.py, conversations.py, documents.py, health.py, projects.py, research.py, sources.py), backend/app/services/ (research_engine.py, relevance_service.py, query_classifier.py, evidence_service.py, contradiction_service.py, ieee_docx.py, gemini_llm.py, search.py), src/app/api/v1/[...path]/route.ts (Proxy), src/components/ResearchWorkspace.tsx (Master UI), src/components/workspace/ (ReportViewer.tsx, SourcesPanel.tsx, EvidencePanel.tsx).

8. Frontend Architecture: Next.js 16 & React 19

Desktop 3-Panel Workspace (SourcesPanel | ReportViewer | EvidencePanel). Interactive [X] citations highlight linked evidence cards in real-time. Mobile viewports (<1024px) utilize a segmented bottom tab navigator with 44px+ touch targets and zero horizontal scroll.

9. Backend Architecture: FastAPI & API Endpoints

FastAPI router modules provide 24+ REST endpoints across /api/v1/auth, /api/v1/research, /api/v1/conversations, /api/v1/projects, /api/v1/documents, and /api/v1/health. All endpoints support standard HTTP methods, bearer authentication, correlation IDs, and structured exception handlers.

10. Database Architecture & Schema Reference

PostgreSQL database tables: users, conversations, messages, projects, research_questions, document_files, document_chunks, sources, research_tasks, claims, evidence_items, contradictions, generated_documents. All tables strictly enforce cascade deletions and foreign keys.

11. Authentication: NextAuth v5 & Google OAuth

NextAuth v5 beta manages frontend OAuth sessions. Upon login, the client calls /api/v1/auth/oauth_sync to create/link the PostgreSQL user record and issue an application JWT token.

12. AI & LLM Engine: Gemini 2.5 Grounding

The LLM is strictly constrained via prompt engineering and JSON schema enforcement. It synthesizes findings strictly from retrieved candidate sources passing the relevance gate, ensuring zero ungrounded hallucinations.

13. Deep Research Pipeline: 7-Stage Orchestrator

Stage 1: Intent & Domain Classification -> Stage 2: Concurrent Multi-Source Retrieval -> Stage 3: Multi-Signal Scoring & Hard Gate -> Stage 4: Evidence Extraction & Grounding -> Stage 5: Contradiction Detection -> Stage 6: Multi-Archetype Synthesis -> Stage 7: IEEE DOCX Document Generation.

14. External Research & Academic APIs

Integrated search providers: OpenAlex API (works & citations), arXiv API (preprints in CS/Physics/Math), PubMed & Europe PMC (biomedical & life sciences), Crossref API (DOI registry), DuckDuckGo (live web news), Wikipedia API (encyclopedic summaries).

15. Source Relevance Scoring & Hard Gate

Composite Score = (Title_Score * 0.35) + (Abstract_Score * 0.25) + (Concept_Intersection * 0.30) + (Exact_Phrase * 0.10) - (Negative_Domain_Penalty). Sources with score >= 0.70 are tagged HIGH; >= 0.48 tagged MODERATE; < 0.48 are hard-rejected and discarded.

16. Evidence Matrix & Grounding System

Evidence items store verbatim quotes, context, page numbers, 'why_relevant' rationales, and qualitative confidence ratings (High / Moderate / Limited). Linked directly to [X] citations in the synthesized report.

17. Answer Generation & Synthesis Structure

Synthesizes structured multi-tiered output: 1. Direct Answer Summary, 2. Key Takeaways, 3. In-Depth Analysis, 4. Methodological Notes and Contradictions.

18. Document Generation: IEEE Two-Column DOCX

IEEEDocumentGenerator formats formal IEEE manuscripts: 24pt bold title, author metadata block, italic abstract, continuous two-column body text, and numbered references.

19. File Uploads & Local Document Extraction

PDF papers uploaded via /api/v1/documents/upload are parsed by PyPDF, segmented by sliding-window chunker, and added to the candidate source pool.

20. User History, Workspaces & Isolation

Users can organize inquiries into Project workspaces and Conversation threads. All queries enforce 'where user_id == current_user.id'.

21. Environment Variables & Secrets Reference

DATABASE_URL, GEMINI_API_KEY, SECRET_KEY, CORS_ORIGINS, NEXTAUTH_SECRET, AUTH_GOOGLE_ID, AUTH_GOOGLE_SECRET, BACKEND_INTERNAL_URL.

22. Local Development Setup from Scratch

1. Clone repo -> 2. Backend: venv, pip install -r requirements.txt, uvicorn app.main:app --port 8000 -> 3. Frontend: npm install, npm run dev -> 4. Open http://localhost:3000.

23. Running in VS Code & Windows Guide

Use PowerShell in VS Code. Configure execution policy if script activation is restricted. Verify ports 3000 and 8000 are open.

24. First-Run Verification Checklist

Verify health check at /api/v1/health, test research query submission, verify source relevance badges, check evidence matrix, test Word document download.

25. Testing Strategy & Automated Benchmark Suites

Pytest test suites in backend/tests: test_docx_generation.py, test_production_research_features.py, test_research_evaluation.py (20-query evaluation benchmark suite).

26. Logging & Observability Guide

Structured logging records request method, path, status, and duration in ms. Request correlation tracked via X-Request-ID.

27. Cloud Deployment Architecture on Railway

Three production services: Frontend (Next.js), Backend (FastAPI Python), and Database (Managed PostgreSQL 16).

28. Production Configuration & Next.js Proxying

Dynamic reverse proxy at src/app/api/v1/[...path]/route.ts forwards requests to backend with 120s timeout and binary buffer streaming.

29. Security, Isolation & Compliance

Parameterized SQL queries, XSS sanitization, secret redaction, and user data isolation across all database operations.

30. Manual Step-by-Step Rebuild Guide (15 Phases)

Comprehensive 15-phase blueprint: Environment -> DB Models -> Middleware -> Auth -> Search Providers -> Relevance Gate -> Classifier -> Evidence Engine -> Gemini LLM -> Pipeline Orchestrator -> IEEE DOCX Builder -> Proxy -> UI Workspace -> Mobile Layout -> Deployment.

31. Recommended Rebuild Order

Strict chronological build order from database foundations to API providers, relevance scorers, LLM synthesis, UI panels, and deployment.

32. Beginner Guide to Core Technical Concepts

Explanations of REST APIs, OAuth 2.0, ORM, Evidence Grounding, and Hard Gating in simple, accessible language.

33. Comprehensive Troubleshooting Guide

Diagnostic tables covering CORS errors, database connection failures, search provider timeouts, and Word document streaming issues.

34. Maintenance & Operations

Procedures for non-destructive database migrations, dependency updates, and AI model version upgrades.

35. Current Known Limitations & Trade-offs

Academic search provider rate limits, LLM synthesis latency (8-15s), and memory considerations for large PDF uploads.

36. Recommended Future Improvements

Redis caching, Celery background worker queues, WebSocket real-time streaming, and Semantic Scholar API integration.

37. Technical Glossary of Terms

Key terminology definitions: Grounding, Hard Gate, IEEE Format, DOI, OpenAlex, PubMed, Relevance Tier.

38. Command Reference & Final Checklist

CLI reference for dev, build, test, and deployment. Final verification checklist confirms complete project reproducibility from zero.